

Git for Developers

Module 4 — Remote Collaboration

GitHub, GitLab, Bitbucket — Push, Pull, Fork, and Contribute

Module 4 — Agenda

Remote Repository Basics

- GitHub, GitLab, and Bitbucket overview
- Remote repositories: `git remote`, `git push`, `git pull`
- Cloning and fetching

Collaborative Workflows

- Fetch vs. Pull — what's the difference?
- Working with multiple remotes: `upstream` and `origin`
- Forking and contributing to open-source projects

Part 1 — Hosting Platforms Overview

GitHub, GitLab & Bitbucket

Platform	Best for	Key features
GitHub	Open source, SaaS teams	Actions CI/CD, Copilot, large ecosystem
GitLab	Self-hosted enterprises	Built-in CI/CD, issues, container registry
Bitbucket	Atlassian shops (Jira, Confluence)	Tight Jira integration, Pipelines CI/CD

All three host Git repositories. Your local Git workflow is identical regardless of platform.

What a Remote Hosting Platform Provides

- **Centralized remote repository** — the `origin` your team pushes to
- **Web UI** for browsing code, history, and diffs
- **Pull Requests / Merge Requests** — code review before merging
- **Issue tracking** — bugs, tasks, feature requests
- **CI/CD pipelines** — automated testing and deployment
- **Access control** — who can read, write, and merge
- **Branch protection rules** — prevent direct pushes to `main`

Part 2 — Remote Repositories

git remote — Managing Remotes

```
# List configured remotes  
git remote -v
```

```
# Add a new remote  
git remote add origin https://github.com/org/repo.git
```

```
# Change a remote URL (e.g., switch from HTTPS to SSH)  
git remote set-url origin git@github.com:org/repo.git
```

```
# Remove a remote  
git remote remove upstream
```

```
# Rename a remote  
git remote rename origin github
```

Understanding `origin`

`origin` is just a conventional alias for the primary remote:

```
$ git remote -v
origin  git@github.com:org/my-project.git (fetch)
origin  git@github.com:org/my-project.git (push)
```

- Created automatically by `git clone`
- Nothing special about the name — you can have any number of remotes
- `origin/main` is a **remote-tracking branch** — your local copy of where `origin`'s `main` was last time you fetched

`git push` — Uploading Commits

```
# Push current branch to the same-named branch on origin
git push

# Push and set the upstream tracking branch (first push)
git push -u origin feature/user-auth

# Push a specific local branch to a specific remote branch
git push origin feature/user-auth:feature/user-auth

# Delete a remote branch
git push origin --delete feature/user-auth

# Force push (rewrites remote history – use with caution!)
git push --force-with-lease
```

Use `--force-with-lease` instead of `--force` — it aborts if someone else has pushed since your last fetch.

git pull — Downloading and Integrating

```
# Fetch from origin and merge into current branch  
git pull
```

```
# Fetch from origin and rebase instead of merge  
git pull --rebase
```

```
# Pull from a specific remote and branch  
git pull origin main
```

```
# Pull all branches from all remotes  
git fetch --all
```

Part 3 — Fetch vs. Pull

Fetch vs. Pull — The Difference

Command	What it does
<code>git fetch</code>	Downloads remote changes — does not modify your working directory or current branch
<code>git pull</code>	Equivalent to <code>git fetch</code> followed by <code>git merge</code> (or <code>git rebase</code> with <code>--rebase</code>)

```
# Fetch: safe, non-destructive – see what's changed before integrating
git fetch origin
git log HEAD..origin/main --oneline    # what's new on the remote?
git diff HEAD origin/main             # review changes
git merge origin/main                  # integrate when ready

# Pull: convenient shortcut
git pull
```

Many experienced developers prefer `git fetch` + manual review + `git merge` for better control.

Tracking Branches

When you clone or push with `-u`, Git sets up a **tracking relationship**:

```
$ git branch -vv
* main      a3f7c21 [origin/main] feat: add authentication
feature b2e1d40 [origin/feature] work in progress
```

- `[origin/main]` means this branch tracks `origin/main`
- `git push` and `git pull` with no arguments use this tracking info
- `git status` shows "Your branch is 2 commits ahead of origin/main"

```
# Set upstream tracking for an existing branch
git branch --set-upstream-to=origin/main main
```

SSH vs. HTTPS Authentication

Method	URL format	Auth mechanism
HTTPS	<code>https://github.com/org/repo.git</code>	Username + token / credential manager
SSH	<code>git@github.com:org/repo.git</code>	SSH key pair

```
# Generate an SSH key
ssh-keygen -t ed25519 -C "jane@example.com"

# Copy public key to clipboard (macOS)
pbcopy < ~/.ssh/id_ed25519.pub

# Test SSH connection to GitHub
ssh -T git@github.com
# Hi jane! You've successfully authenticated.
```

Add the **public** key (`~/.ssh/id_ed25519.pub`) to GitHub/GitLab under Settings
→ SSH Keys.

Part 4 — Multiple Remotes & Forking

Working with Multiple Remotes

Common pattern: **origin** (your fork) + **upstream** (original project)

```
# You've forked a project on GitHub – clone your fork
git clone git@github.com:jane/open-source-project.git

# Add the original project as 'upstream'
git remote add upstream git@github.com:original-org/open-source-project.git

$ git remote -v
origin      git@github.com:jane/open-source-project.git (fetch)
origin      git@github.com:jane/open-source-project.git (push)
upstream    git@github.com:original-org/open-source-project.git (fetch)
upstream    git@github.com:original-org/open-source-project.git (push)
```

Keeping Your Fork in Sync

```
# Fetch the latest changes from the original project
git fetch upstream

# Merge upstream/main into your local main
git checkout main
git merge upstream/main

# Push the updated main to your fork
git push origin main
```

Always sync your fork before starting new work — reduces conflicts later.

Forking & Contributing — The Open Source Workflow

1. Fork the repo on GitHub (creates your personal copy)
2. Clone your fork locally
3. Create a feature branch: `git checkout -b feat/my-fix`
4. Make changes, commit, push to your fork
5. Open a Pull Request: your fork/branch → original repo/main
6. Maintainer reviews, requests changes, approves, and merges
7. Sync your fork with upstream after merge

Pull Requests — Best Practices

- **One PR per feature or fix** — keep it focused and reviewable
- **Write a clear description** — explain the *why*, link to issues
- **Keep PRs small** — easier to review, faster to merge
- **Respond to review comments** promptly
- **Don't force-push** after a PR is open (unless rebasing is required)
- **Squash commits** if the branch history is messy (maintainer may do this)

Summary — Remote Collaboration

Command	What it does
<code>git remote add origin <url></code>	Add a remote
<code>git remote -v</code>	List remotes
<code>git push -u origin <branch></code>	Push and set tracking
<code>git fetch</code>	Download remote changes (no merge)
<code>git pull</code>	Fetch + merge
<code>git pull --rebase</code>	Fetch + rebase
<code>git remote add upstream <url></code>	Add original project as upstream
<code>git fetch upstream</code>	Get latest changes from upstream